“Intelligent Digital Library Search & Management System (IntelliLib)”

Project Report submitted for the partial fulfilment of grade for the course

Data Structures and Algorithms - 3 (25CS2103E)

in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

by

NITHISH S : 2520030346

METUKU NIHAL : 2520030124

SAI SATHWIK : 2520030328

Team Number: 2

Section: 8

Under the Guidance of

Dr. V. Sireesha

Associate Professor

Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

2026-27

Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

Declaration

The Project Report entitled "Intelligent Digital Library Search & Management System (IntelliLib)" is a record of bonafide work carried out by Nithish S – 2520030346, Metuku Nihal – 2520030124, and Sai Sathwik – 2520030328, submitted in partial fulfilment of the requirements for the course Data Structures and Algorithms - 3 (25CS2103E) in the Bachelor of Technology programme in Computer Science and Engineering during the academic year 2026-27.

We declare that the work presented in this report is based on our own design, implementation, testing, and analysis of the IntelliLib system. The implementation combines desktop application development, cloud deployment, a central relational database, real camera-based QR/barcode scanning, and data-structure-driven search. Wherever external documentation, libraries, textbooks, or technical references were used, they have been acknowledged in the References section. The results embodied in this report have not been copied from any other department, university, or institute.

Nithish S – 2520030346

Metuku Nihal – 2520030124

Sai Sathwik – 2520030328

Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

CERTIFICATE

This is to certify that the mini project report entitled "Intelligent Digital Library Search & Management System (IntelliLib)" is a bonafide work done and submitted by Nithish S – 2520030346, Metuku Nihal – 2520030124, and Sai Sathwik – 2520030328 in partial fulfilment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY in the Department of Computer Science and Engineering, during the academic year 2026-27.

The work has been carried out under the guidance of Dr. S. Vinay Kumar, Associate Professor, Department of Computer Science and Engineering. The report describes the analysis, design, implementation and validation of a live physical-library management system using JavaFX, Spring Boot, PostgreSQL, cloud deployment, camera-based code scanning and classical data structures and algorithms.

Signature of the Guide
(Dr. S. Vinay Kumar)

Signature of the HOD

Signature of the External Examiner

ACKNOWLEDGEMENT

We express our sincere gratitude to Dr. S. Vinay Kumar, Associate Professor, Department of Computer Science and Engineering, for his guidance, encouragement, and constructive suggestions throughout the development of IntelliLib. His support helped us convert the initial concept of a digital library search system into a complete two-role desktop application that reflects the operational needs of a physical library. The discussions on data structures, software design, testing discipline, and presentation of results were particularly valuable in improving both the implementation and this report.

We also thank Dr. P. Venkateshwara Rao, Associate Professor and Head of the Department of Computer Science and Engineering, along with the faculty members of KLH Hyderabad, for providing the academic environment and resources required to complete the project. We are grateful to the Department for encouraging practical implementation of Data Structures and Algorithms through a project that connects classroom concepts with a real-world information-management problem.

Finally, we thank our teammates, classmates, and everyone who contributed feedback during testing. The project required repeated refinement of the user experience, camera scanning, cloud deployment, database schema, authentication, search behaviour, and circulation workflow. The cooperation within the team made it possible to integrate these parts into one coherent system and document the learning obtained from the process.

Nithish S – 2520030346

Metuku Nihal – 2520030124

Sai Sathwik – 2520030328

Abstract

IntelliLib is a live desktop-based library search and circulation system designed for a real physical college library. The project addresses two connected problems: students need a fast way to discover relevant library resources and request them, while librarians need a reliable method to approve requests, identify students, track exact physical copies, issue and return books, and maintain one consistent source of truth. The system therefore combines information retrieval, library operations, identity scanning, transaction processing, and real-time communication in one application.

The solution uses a Java 21 and JavaFX desktop client with only two roles: Student and Administrator. Students enter through an in-app camera scanner that reads the QR or barcode on a registered college ID card. After authentication, they can search by title, author, ISBN, category, or keyword, receive prefix suggestions, view related resources, submit borrow requests, and track approvals, loans, and notifications. Administrators use secure email/password authentication to manage students, books and exact physical copies; approve or reject requests; and perform issue and return operations by scanning the student ID and the barcode of the physical copy. The backend is implemented using Spring Boot and exposes REST and WebSocket interfaces. Neon PostgreSQL provides central durable storage, while Render hosts the backend so the system remains available even when a development laptop is switched off.

Classical data structures and algorithms are integrated beneath the user interface. Hash-based indexes support exact ISBN and token lookup, a Trie provides live autocomplete, an AVL tree maintains alphabetical title access, a graph represents related resources, KMP supports substring matching, and edit distance provides bounded typo recovery. PostgreSQL transactions and row locking protect issue/return integrity, while Flyway manages schema evolution. The production database intentionally starts without generated books or fake dashboard statistics; real students, books and copy barcodes are imported or enrolled by the administrator. The resulting project demonstrates how DSA concepts can be combined with modern desktop, database, security and cloud technologies to build a practical, scalable and explainable library system.

Table of Contents

Contents	Page No.
Acknowledgement	
Abstract	
Table of Contents	
List of Figures and Tables	
1. INTRODUCTION	1
1.1 Overview	1
1.2 Background and Motivation	2
1.3 Problem Statement	3
1.4 Proposed Solution and Scope	4
1.5 Objectives and Contributions	5
1.6 User Journey and Report Organization	6
2. SYSTEM REQUIREMENTS	7
2.1 Stakeholders and Roles	7
2.2 Student Functional Requirements	8
2.3 Administrator Functional Requirements	9
2.4 Non-Functional Requirements	10
2.5 Hardware and Software Requirements	11
2.6 Data, Security and Traceability Requirements	12
3. SYSTEM DESIGN	13
3.1 High-Level Architecture	13
3.2 Layered Design	14
3.3 Desktop Client Design	15
3.4 Backend and API Design	16
3.5 Database Design	17
3.6 Data Integrity and Physical Copy Model	18
3.7 Authentication and Security Design	19
3.8 Borrow Request State Model	20
3.9 Issue Workflow	21
3.10 Return Workflow	22
3.11 Search, Recommendation and Deployment Design	23
4. IMPLEMENTATION	24
4.1 Technology Stack and Project Structure	24
4.2 Camera Scanner	25
4.3 Student Authentication and Sessions	26
4.4 Administrator Authentication and Management	27
4.5 Hash-Based Search Indexes	28
4.6 Trie and AVL Indexes	29
4.7 Graph-Based Related Resources	30
4.8 String Matching and Typo Recovery	31
4.9 Requests, Notifications and Real-Time Updates	32
4.10 Atomic Circulation, Import, Migrations and Packaging	33
5. RESULTS	34
5.1 Windows Build and Packaging	34
5.2 Camera Scanning and Validation	35
5.3 Cloud Deployment	36
5.4 Live Database and Verification Matrix	37
6. CHALLENGES AND LIMITATIONS	38
6.1 Development Challenges and Solutions	38
6.2 Current Limitations and Risk Management	39
7. CONCLUSION AND FUTURE WORK	40
7.1 Conclusion	40
7.2 Future Work	41
7.3 Skills Learned	42
REFERENCES	43

List of Figures and Tables

List of Figures

Fig. 1.1 Student journey through IntelliLib 6
Fig. 3.1 High-level live system architecture 13
Fig. 3.2 Layered architecture of IntelliLib 14
Fig. 3.3 Entity relationship model 17
Fig. 3.4 Student card and session security flow 19
Fig. 3.5 Borrow request and loan state model 20
Fig. 3.6 Issue book workflow 21
Fig. 3.7 Return book workflow 22
Fig. 3.8 DSA components used by search 23
Fig. 5.1 Portable Windows build output 34
Fig. 5.2 In-app camera scanner with privacy redaction 35
Fig. 5.3 Successful Render deployment connected to Neon 36
Fig. 5.4 Administrator dashboard using live database counts 37

List of Tables

Table 2.1 Stakeholders and responsibilities 7
Table 2.2 Student functional requirements 8
Table 2.3 Administrator functional requirements 9
Table 2.4 Non-functional requirements 10
Table 2.5 Hardware and software requirements 11
Table 2.6 Requirements traceability matrix 12
Table 3.1 REST API groups 16
Table 3.2 Core data entities and integrity rules 18
Table 4.1 Technology stack 24
Table 4.2 DSA roles and complexity summary 31
Table 5.1 Verified implementation results 37
Table 6.1 Challenges and applied solutions 38

1. INTRODUCTION

1.1 Overview

Libraries remain one of the most important shared knowledge resources in an academic institution, but the physical act of finding, requesting, issuing and returning a book still depends on accurate records and smooth coordination between students and library staff. A conventional catalogue may tell a student that a title exists, yet it does not automatically guarantee that a physical copy is available, that the correct student has requested it, or that the copy handed over at the desk is the same copy recorded in the system. IntelliLib was designed to treat these concerns as one connected problem rather than as independent screens.

The project began from a DSA-oriented requirement: efficiently manage and retrieve thousands of books, journals, magazines and research papers using title, author, ISBN, category and keyword search. The implementation was expanded into a complete physical-library workflow while keeping the data-structure focus underneath the user interface. Students interact with a clean desktop application, authenticate by scanning their registered college ID, search the real catalogue, request a book and receive live updates. Administrators approve requests, maintain real student and book records, and issue or return the exact physical copy using its barcode.

A central design principle is that the production system must not pretend. IntelliLib does not pre-populate generated books, fake borrowers, fake popularity counts or fake analytics. A fresh production database contains the administrator account and schema, while real library data is introduced through controlled import or registration. This makes dashboard counts, availability, borrowing history and recommendations consequences of actual operations. The resulting system therefore combines the algorithmic focus of DSA with the correctness requirements of a real database-backed application.

1.2 Background and Motivation

A student searching a large physical library faces two different kinds of complexity. The first is information complexity: a relevant resource may be known only by a partial title, an author name, a subject keyword or an approximate spelling. The second is operational complexity: even after the right title is found, the student needs to know whether a copy is available, whether a request can be placed, whether the request has been approved and when the book is actually issued. Sequentially scanning records or relying on disconnected spreadsheets becomes increasingly inefficient as the catalogue and user base grow.

Classical data structures provide natural solutions to several parts of this problem. Hash tables are well suited to exact identifiers such as ISBNs and normalized tokens. Tries support prefix-based autocomplete, which is useful when a student remembers only the beginning of a title or author name. Balanced search trees support ordered access, while graph relationships can connect books that share an author, category or keyword. String matching and edit-distance techniques can further improve retrieval when the query is a substring or contains a small spelling error. These techniques are meaningful because they directly improve a real user action instead of appearing as isolated classroom demonstrations.

The motivation for the production architecture came from the same principle of realism. If the server runs only on a developer laptop, the system stops when that laptop is shut down. IntelliLib therefore separates the Windows desktop client from the central backend. Render hosts the Spring Boot service, Neon provides PostgreSQL storage, and desktop clients communicate over HTTPS and WebSocket connections. This enables a student PC and an administrator PC to see the same live state without installing PostgreSQL, Maven or Docker on every machine.

1.3 Problem Statement

The primary problem addressed by IntelliLib is the lack of a single, dependable workflow that joins efficient resource discovery with correct physical circulation. A student should not need to search one system, contact a librarian in another channel, and then rely on a separate register to confirm whether the desired copy was issued. Likewise, an administrator should not have to manually reconcile requests, student identities, book titles and physical copy identifiers after the event.

From the search perspective, a naïve implementation that scans every record for every query does not scale well and produces a weak interactive experience. It also fails to exploit the fact that library data contains multiple useful keys: ISBNs are exact, titles and authors are naturally prefix-searchable, categories and keywords create clusters, and related books form relationships. The project therefore requires data structures that can represent these access patterns explicitly and a ranking process that combines them into a useful search response.

From the circulation perspective, correctness is more important than visual convenience. The system must prevent invalid state changes such as issuing a book without an approved request, issuing an already issued copy, recording the same physical copy as simultaneously held by two students, or returning a copy against the wrong borrower. It must also distinguish a bibliographic title from a physical copy. “Introduction to Algorithms” may be one book record, while LIB-000001 and LIB-000002 are separate copies with separate availability states. The problem statement can therefore be summarized as follows: design and implement a live, secure and data-structure-driven library system that supports fast discovery, exact copy tracking, controlled request approval, transactional issue/return operations and consistent real-time information for both students and administrators.

1.4 Proposed Solution and Scope

IntelliLib proposes a two-role desktop application supported by one central cloud backend and database. The Student role begins with camera-based scanning of the college ID. After authentication, the user can search the catalogue, view availability, request a resource, see request status, view active and historical loans, and receive notifications. The Administrator role uses email/password authentication and provides dashboard counts, request approval or rejection, student registration/import, book and copy management, issue/return scanning, loan history and audit activity.

The scope intentionally focuses on a physical college library. IntelliLib is not an e-book reader, social network, payment platform, chatbot or public marketplace. It does not attempt to replace institutional identity systems or add unrelated features simply to increase the number of screens. Instead, the project concentrates on accurate resource discovery and circulation. This narrower scope allows the underlying DSA components, database transactions, security rules and live communication to be integrated properly.

The system supports real CSV imports for students, bibliographic book records and physical copy barcodes. Book titles and physical copies are modelled separately so that one title can have many copies. Requests move through controlled states, and issue/return operations are executed inside database transactions. The desktop client is packaged as a Windows application image with its own Java runtime so ordinary client PCs do not require developer tools. Docker remains a deployment/build mechanism for the backend rather than a user-facing requirement.

1.5 Objectives and Contributions

The objectives of the project were defined around measurable behaviours rather than around a list of technologies. The first objective is to reduce the work required to discover a relevant resource by combining exact lookup, token-based retrieval, autocomplete, ordered access, related-resource discovery and bounded typo recovery. The second objective is to ensure that the status shown to a user reflects the central database, particularly the number of available physical copies. The third objective is to provide a clear student request workflow and an administrator approval workflow that can operate across different PCs.

A further objective is to make circulation safe. The administrator must scan both the student identity and the exact physical copy, and the server must validate the relationship between the student, approved request, book and copy before a loan is created. The reverse checks apply to returns. These operations must be atomic so that partial failures do not leave a loan created without changing copy status, or vice versa. Security objectives include hashing administrator passwords, avoiding storage of raw student card codes, hashing session tokens in the database and enforcing role checks on server endpoints.

The main contribution of IntelliLib is therefore the integration of DSA concepts into a coherent production-style workflow. The user interface does not expose algorithm names because a real student should see “Search” and “Related Books,” not “Trie” or “Graph.” However, the implementation retains those structures internally and documents them for academic evaluation. The project also demonstrates practical lessons that are normally outside isolated DSA exercises: schema migrations, deployment reliability, cloud configuration, scanner hardware integration, validation errors, live notifications, packaging and the distinction between demo data and real operational data.

1.6 User Journey and Report Organization

The student journey is deliberately short. After choosing Student, the in-app camera scanner appears immediately. A recognized and enrolled college ID creates a student session. From the home screen, the student can search, open a resource, submit a request, wait for an approval notification, collect the resource after administrator verification, and later return the same physical copy. This sequence is illustrated in Fig. 1.1. The administrator journey runs in parallel: secure login, request review, approval, student scan, copy scan, transactional issue, and later transactional return.



Fig. 1.1 Student journey through IntelliLib

The remainder of this report is organized to move from requirements to evidence. Chapter 2 identifies functional, non-functional, hardware, software, data and security requirements. Chapter 3 presents the architecture, database, security, request lifecycle, circulation workflows, search design and deployment model. Chapter 4 explains how these designs were implemented, including the camera scanner, authentication, DSA indexes, string matching, notifications, transactions, imports, migrations and Windows packaging. Chapter 5 records verified results from the build, scanner, deployment and live database. Chapter 6 discusses the development challenges and current limitations. Chapter 7 concludes the work, identifies future improvements and summarizes the skills learned during development.

2. SYSTEM REQUIREMENTS

2.1 Stakeholders and Roles

IntelliLib has only two application roles: Student and Administrator. This is an intentional simplification that matches the target college-library workflow. A separate “counter” role was rejected because issue and return are already administrator responsibilities and a third role would add unnecessary permission complexity. The backend nevertheless distinguishes each authenticated principal and checks role requirements before allowing privileged operations.

Table 2.1 Stakeholders and responsibilities

Stakeholder	Primary responsibilities	Authentication
Student	Search catalogue; view availability; request books; view approvals, loans and notifications.	Registered college ID QR/barcode scanned by camera.
Administrator	Manage students/books/copies; approve/reject requests; issue/return books; review loans and audit activity.	Email and password with server-side BCrypt verification.
Library/Institution	Provides real catalogue, student identifiers, physical copy labels and operational rules.	Operational authority; not a separate IntelliLib role.
System/Cloud Services	Maintain API availability, persistent data, migrations and live message delivery.	Service credentials and environment secrets.

The student is treated as the borrower and should not be exposed to administrative controls. The administrator is responsible for data quality and physical circulation. These boundaries simplify the user interface and also create clear authorization rules on the server. The same desktop application can display either role, but the session returned by the backend contains a role and each endpoint verifies the expected role before data is returned or modified.

2.2 Student Functional Requirements

Student requirements were derived from the expected journey of a real borrower. The most important requirement is that authentication begins with the physical college ID rather than a typed demo token. The scanner is presented inside IntelliLib so the user does not leave the application. Once authenticated, the student should be able to discover resources using more than one metadata field and should see honest empty states when the library has no data or when recommendation history is insufficient.

Table 2.2 Student functional requirements

ID	Requirement	Expected behaviour
SFR-01	College ID scanning	Open an in-app camera view and decode a supported QR/barcode format.
SFR-02	Student authentication	Match the scanned code to an active enrolled student and create a session.
SFR-03	Multi-field search	Search title, author, ISBN, category and keywords.
SFR-04	Autocomplete	Suggest title/author prefixes while typing.
SFR-05	Book details	Show bibliographic metadata, total copies and available copies.
SFR-06	Borrow request	Allow a request only when an available copy exists and no active duplicate request exists.
SFR-07	Request tracking	Show Requested, Approved, Rejected, Issued and Expired states.
SFR-08	Loan tracking	Show exact issued copy barcode, issue date, due date, return date and effective status.
SFR-09	Notifications	Show approval, issue and return notifications, including live updates.
SFR-10	Recommendations	Show popular books from real loans and personalized suggestions from the student's own activity.

A critical negative requirement is that the student application must not invent data. If no books are imported, the catalogue is empty. If there is no borrowing history, the popular section explains that history is not yet available. If the student has no meaningful views or borrowing activity, the recommendation area remains empty rather than displaying synthetic personalization.

2.3 Administrator Functional Requirements

The administrator side must support both information management and physical circulation. Because incorrect administrative actions can affect inventory and borrowers, each operation is validated on the server rather than trusted simply because the client displayed a button. Student registration, book import, copy assignment, approval and circulation therefore become controlled domain operations rather than direct database edits from the desktop application.

Table 2.3 Administrator functional requirements

ID	Requirement	Expected behaviour
AFR-01	Secure sign-in	Authenticate administrator email/password and create an administrator session.
AFR-02	Live dashboard	Display counts obtained directly from the central database.
AFR-03	Student management	Register by scanning ID, import CSV, list students and activate/deactivate accounts.
AFR-04	Book management	Add/search books and import real catalogue metadata.
AFR-05	Copy management	Track each physical barcode separately and import copy data.
AFR-06	Request decision	Approve/reject pending borrow requests with optional notes.
AFR-07	Issue book	Scan student, verify approval, scan exact copy, create loan atomically.
AFR-08	Return book	Scan student and copy, verify active loan, complete return atomically.
AFR-09	Loan history	Review issued, returned and overdue loans.
AFR-10	Audit activity	Record important administrative and circulation actions with workstation context.

The dashboard itself is intentionally simple. Counts such as Books, Physical Copies, Available, Students, Issued, Pending Requests and Overdue are aggregate SQL results. No artificial values are inserted merely to make the screen appear populated. This requirement is important because the dashboard is used to assess whether the live database reflects the actual library state.

2.4 Non-Functional Requirements

Non-functional requirements determine whether the system remains usable and trustworthy after individual features are implemented. IntelliLib prioritizes correctness, clarity and maintainability over excessive visual effects. The desktop interface uses a consistent light visual language, custom in-app overlays, readable typography and clear empty/error states. Network operations are performed away from the JavaFX UI thread so that a slow cloud wake-up does not permanently freeze the interface.

Table 2.4 Non-functional requirements

Category	Requirement
Performance	Interactive search should use in-memory indexes rather than repeated full-table scans. Availability refreshes should be targeted to the affected book.
Usability	Student and administrator journeys must be understandable without exposing internal algorithm names.
Reliability	Issue/return operations must be transactional and reject invalid transitions.
Scalability	Search structures should be rebuilt from the database and support growth in catalogue size without changing the user model.
Security	Passwords are hashed; raw card codes are not stored; sessions expire; role checks are server-side.
Maintainability	Database schema evolution is versioned with Flyway and deployment configuration is separated from source secrets.
Portability	Client PCs run a packaged IntelliLib application with a bundled runtime; no local database or Docker requirement.
Availability	Cloud hosting keeps the backend independent of the development PC, subject to Free-tier cold-start limits.
Data integrity	Unique identifiers, foreign keys, partial unique indexes and transactions enforce inventory consistency.
Observability	Audit records and deployment logs provide evidence for important administrative and operational events.

These requirements also shape testing. A successful screen rendering is not enough if a copy can be issued twice or if a student can access administrator endpoints. For that reason, design and test discussion in later chapters treats state transitions, role boundaries, data constraints and deployment behaviour as first-class quality criteria.

2.5 Hardware and Software Requirements

The production architecture separates end-user requirements from development/deployment requirements. A student or administrator PC only needs Windows, a supported webcam and network access to the configured backend. The packaged Java runtime is included with IntelliLib. Development and server build tooling is therefore not imposed on ordinary library PCs.

Table 2.5 Hardware and software requirements

Area	Requirement / technology	Purpose
Client hardware	Windows PC; webcam; keyboard/mouse; network connection	Run IntelliLib and scan ID/book codes.
Student identity	College ID containing a unique QR/barcode value	Primary scan-based identity input.
Book inventory	Unique barcode/QR label for each physical copy	Exact copy identification during issue/return.
Client software	Java 21 runtime bundled by jpackage; JavaFX 21	Desktop UI and local scanner integration.
Backend	Spring Boot 3.5.x on Java 21	REST API, validation, services and WebSocket messaging.
Database	PostgreSQL hosted by Neon	Durable central data and transactions.
Cloud hosting	Render Web Service using Docker image build	Public backend independent of developer PC.
Schema management	Flyway migrations	Versioned creation and evolution of database schema.
Scanning libraries	webcam-capture 0.3.12 and ZXing 3.5.3	Camera access and multi-format QR/barcode decoding.
Build tools	Maven 3.9.x, JDK 21, Git/GitHub	Compile, package, version and deploy the project.

For zero-cost pilot deployment, Render Free and Neon Free were selected. This satisfies the academic requirement that the system can be demonstrated as a real cloud-connected application without requiring the developer laptop to remain powered on. However, the limitations of a free sleeping web service are explicitly documented later and should not be confused with an institutional service-level guarantee.

2.6 Data, Security and Traceability Requirements

Real data requirements are as important as functional requirements. Students are represented by a student code, name and optional academic/contact fields together with a securely derived card hash. Books represent bibliographic titles, while physical copies use unique barcodes and explicit statuses. The system must be capable of starting with zero production books and zero production students, then becoming useful through administrator-controlled imports and enrollment.

Security requirements include a secret card pepper stored in the deployment environment rather than in source control, BCrypt password hashing for administrators, random session tokens, hashed session-token storage, session expiry and server-side role validation. Raw student card codes are used transiently for matching and are not stored directly in the users table. Production secrets such as the Neon connection string, card pepper and administrator password are configured in the cloud environment rather than committed to the public project repository.

Table 2.6 Requirements traceability matrix

Requirement	Design element	Verification evidence
Real student scan	In-app webcam + ZXing + hashed card lookup	Camera preview and server rejection of unenrolled card.
Live central state	Render API + Neon PostgreSQL	Successful cloud deployment and live admin dashboard.
No fake metrics	SQL dashboard aggregates; no production seed data	Dashboard initially displays zero real records.
Exact copy tracking	book_copies table with unique barcode	Issue/return design uses scanned copy barcode.
Safe circulation	Transactional service + row locks + state checks	Code-level transaction design and database constraints.
Fast search	Hash, Trie, AVL, Graph, KMP, Edit Distance	SearchEngine rebuilds indexes from live database.
Role separation	Student/Admin sessions and endpoint checks	AuthService requireStudent/requireAdmin model.

3. SYSTEM DESIGN

3.1 High-Level Architecture

IntelliLib follows a client-server architecture. The JavaFX desktop client is responsible for interaction, camera preview, code decoding, presentation and local configuration of the server address. The Spring Boot backend owns business rules, sessions, request decisions, circulation transactions, search services and real-time notifications. Neon PostgreSQL provides the central source of truth. This separation prevents each desktop PC from becoming an independent database and ensures that a change made by the administrator is visible to students through the same backend.

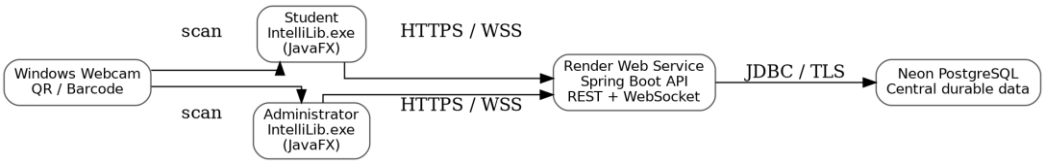


Fig. 3.1 High-level live system architecture

HTTPS is used for request/response communication and WebSockets are used for live events. The client never connects directly to PostgreSQL. This reduces the attack surface and keeps database credentials off student and administrator PCs. It also allows the backend to validate every state-changing operation before data is committed.

3.2 Layered Design

The design separates presentation, application, service, search and persistence responsibilities. The presentation layer contains JavaFX screens, tables, cards and custom overlays such as the camera scanner and error sheets. The application layer maps HTTP requests to typed request/response models and performs authentication checks. The service layer contains domain operations such as student creation, request decisions, issuing, returning and notifications. Search is treated as a dedicated in-memory subsystem that is rebuilt from durable database data. Persistence uses PostgreSQL through JdbcTemplate and Flyway.

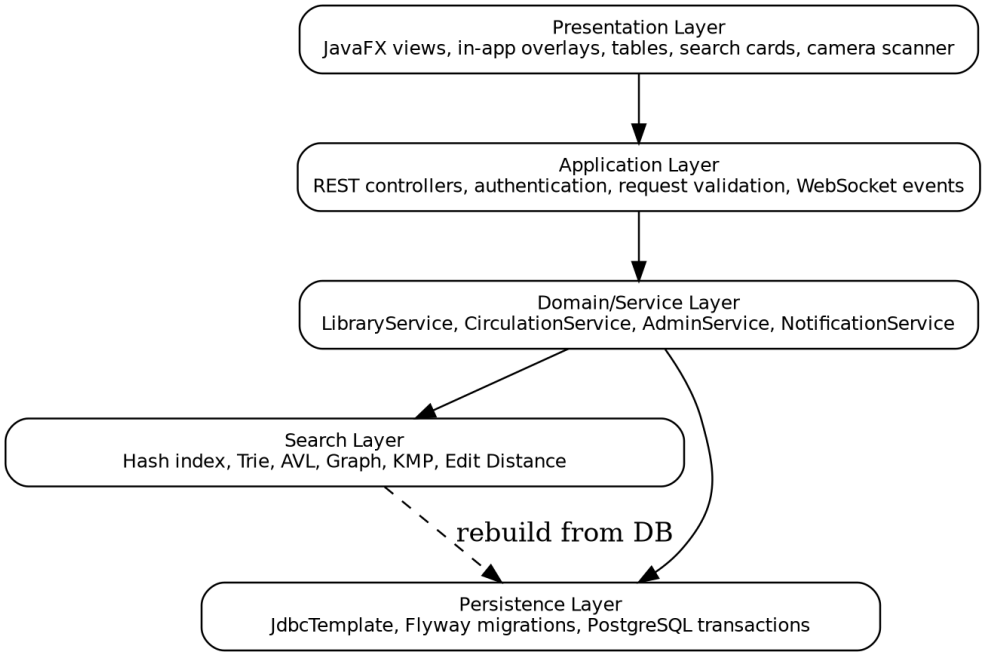


Fig. 3.2 Layered architecture of IntelliLib

The layered approach is important for correctness. For example, the Issue button in JavaFX does not directly update a copy record. It calls a server endpoint, which requires an administrator session and delegates to CirculationService. The service locks and validates database rows, changes request and copy state, creates the loan and then emits a notification. Each layer therefore has a limited responsibility and the most important rules remain on the server where a modified client cannot bypass them.

3.3 Desktop Client Design

The desktop client uses a single JavaFX application shell with role-specific navigation. The landing screen exposes Student and Administrator entry points and a Server Settings option. Choosing Student immediately opens the camera scanner because the intended authentication method is the physical college card. Choosing Administrator opens an email/password form. After login, a sidebar presents the functions appropriate to the role. This design avoids multiple executables while still preserving a clear separation of capabilities.

Long-running network requests are executed on daemon worker threads and UI updates are returned to the JavaFX application thread. This prevents HTTP calls or a sleeping cloud service from blocking the event loop. Custom overlays are used for camera scanning, book details, confirmation and error messages instead of native platform dialogs. The camera overlay includes a camera selector when multiple devices are detected, a live preview, a visual scan frame, retry controls, status text and cancellation.

The client stores only a configurable backend base URL and workstation identifier locally. Authentication state is held in the application session and supplied to the API using a session-token header. The packaged Windows application is produced as an app-image with its own runtime, containing IntelliLib.exe, an app directory and runtime directory. This design allows the complete folder to be copied to another PC without installing Java, Maven, Docker or PostgreSQL.

3.4 Backend and API Design

The Spring Boot backend is organized around REST controllers and services. Authentication endpoints create sessions; book endpoints provide search, suggestions, related resources and student activity; student endpoints expose the authenticated user's requests, loans and notifications; and administrator endpoints provide dashboard, request decision, scan verification, issue/return, imports and activity. The controller layer remains relatively thin so the domain rules are testable and reusable inside services.

Table 3.1 REST API groups

API group	Representative operations	Authorization
/api/auth	Card login; admin login; logout	Public login endpoints; logout uses session if present.
/api/books	Search; suggestions; related; popular; recommended; book view	Search may be public/role-aware; personalized operations use student session.
/api/me	Borrow request; my requests; my loans; notifications	Student session required.
/api/admin	Dashboard; requests; users; books; copies; issue; return; loans; activity	Administrator session required.
/api/health	Service health check	Used by client and Render health monitoring.

Request and response data are represented by typed records such as SessionUser, BookSummary, BorrowRequestView, LoanView, Dashboard, StudentScanResult and CirculationResult. This reduces ambiguous JSON structures and keeps the desktop and backend contract understandable. Error handling returns human-readable messages that the client presents in its own in-app overlay.

3.5 Database Design

The relational schema represents identity, catalogue, physical inventory, requests, loans, activity and audit information. A student and an administrator share the users table but are separated by a role field and different authentication data. A book represents bibliographic information. Each physical copy belongs to one book and has its own unique barcode and status. Borrow requests connect users to books; loans connect users, books, requests and exact physical copies.

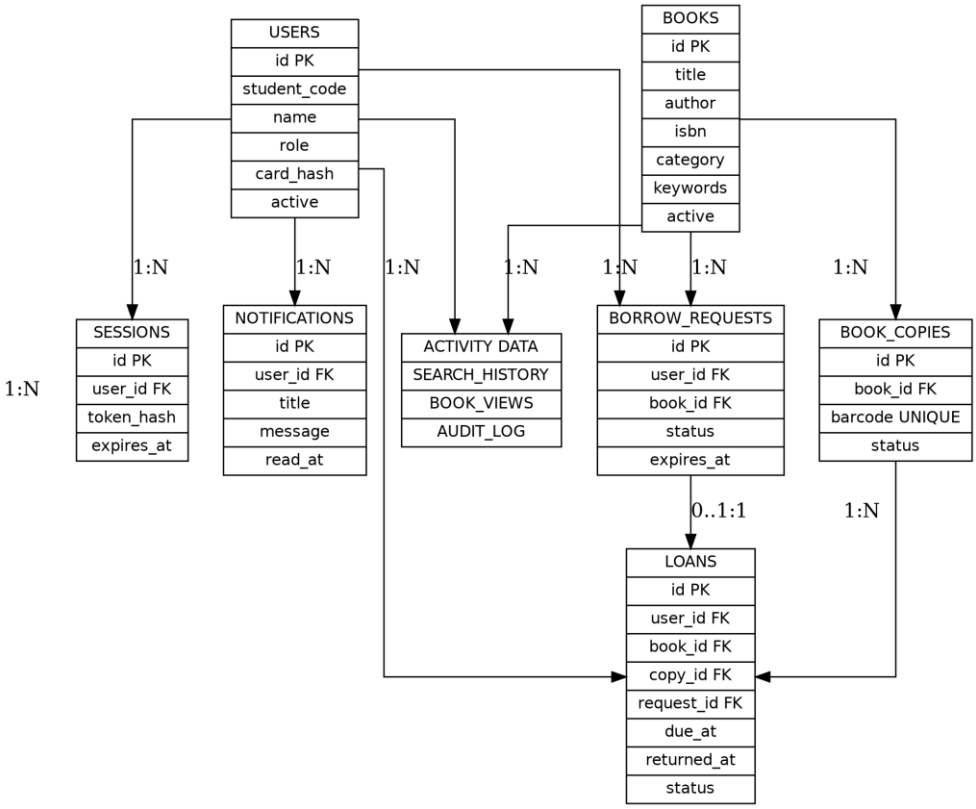


Fig. 3.3 Entity relationship model of the central database

Supporting tables capture sessions, notifications, search history, book views and audit records. Foreign keys prevent orphaned relationships, while indexes support common access paths. Schema migrations are versioned with Flyway. The production deployment shown during testing created and validated the Flyway schema history automatically, which makes the database reproducible on a new Neon project without manual table creation.

3.6 Data Integrity and Physical Copy Model

A major design decision is the separation between a title and a copy. If a library owns three copies of the same ISBN, the books table stores the title once and book_copies stores three rows. Each copy has a unique barcode and status such as AVAILABLE, ISSUED, LOST or ARCHIVED. The issue and return workflows operate on the copy barcode, not only on the title or ISBN. This eliminates ambiguity and makes availability a count of actual copy rows rather than a manually maintained integer.

Table 3.2 Core data entities and integrity rules

Entity	Important rule
users	student_code and card_hash are unique; only active students can use card login.
books	ISBN is unique when present; inactive books are excluded from active search rebuilds.
book_copies	barcode is unique; each row belongs to exactly one book.
borrow_requests	one student cannot hold multiple simultaneous REQUESTED/APPROVED requests for the same book.
loans	a physical copy cannot have more than one active unreturned loan.
sessions	raw tokens are not stored; token hashes are unique and expire.
notifications	belong to one user and preserve the message history.
audit_log	records actor, action, entity and workstation context for important operations.

The schema also uses partial unique indexes where the uniqueness rule depends on state. For example, an active request uniqueness index applies only to REQUESTED and APPROVED rows, allowing historical rejected or completed requests to remain. Similarly, the active-loan uniqueness rule applies only while returned_at is null. These database rules provide a second line of protection even if application code changes later.

3.7 Authentication and Security Design

Student and administrator authentication deliberately use different mechanisms. An administrator supplies an email and password. The password is verified with BCrypt configured with strength 12; plaintext passwords are never stored. A student presents a code read from the college ID card. Before storage or comparison, the code is normalized and processed with HMAC-SHA256 using a secret APP_CARD_PEPPER held in the deployment environment. The users table therefore stores a keyed card hash rather than the raw card value.

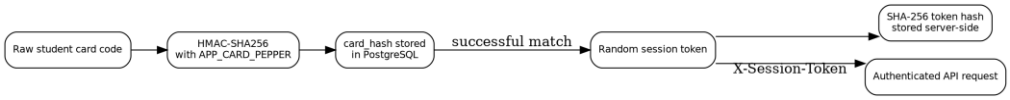


Fig. 3.4 Student card and session security flow

After successful authentication, the server generates a cryptographically random session token. Only the SHA-256 hash of that token is stored in the sessions table. The raw token is returned to the client and supplied in subsequent API requests. Student sessions have a shorter lifetime than administrator sessions. The server checks both session validity and role before serving protected endpoints. This design reduces the impact of database disclosure because neither administrator passwords, raw student card codes nor raw session tokens are directly stored.

3.8 Borrow Request State Model

The request and loan lifecycle is modelled as explicit states rather than as loosely related boolean flags. A student first creates a REQUESTED record for a book. The administrator may APPROVE or REJECT it. Approved requests receive an expiry time so a reservation does not remain indefinitely. When the correct student and copy are scanned, the request becomes ISSUED and a loan is created. A loan can become effectively OVERDUE when its due date passes and finally becomes RETURNED when the exact physical copy is returned.

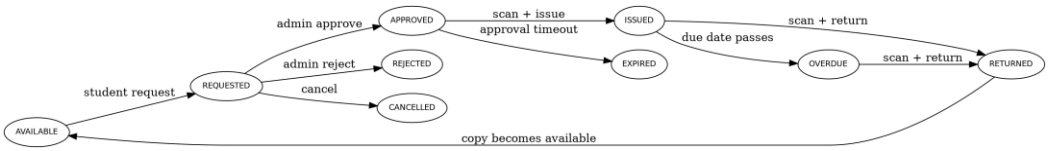


Fig. 3.5 Borrow request and loan state model

The state model makes invalid transitions visible. There is no path from a rejected request directly to issue, and an expired approval cannot be treated as a valid collection. The database and service layer additionally check physical copy state and active-loan constraints. This combination of state-machine reasoning and relational constraints is essential for maintaining consistency when different users act at different times.

3.9 Issue Workflow

Issuing a book connects identity, approval and physical inventory, so it is treated as one controlled server operation. The administrator scans the student ID and the server returns only active, unexpired approved requests for that student. The administrator then scans the exact physical copy barcode. The client checks that the scanned copy belongs to an approved title before the final issue request is submitted.

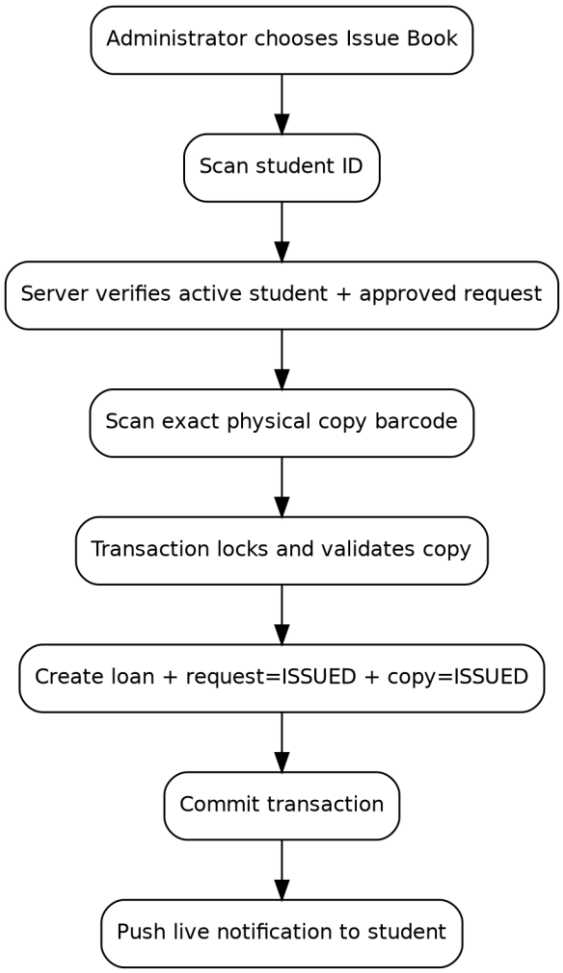


Fig. 3.6 Issue book workflow

The backend repeats all critical checks inside one transaction: resolve the student, lock the copy, require AVAILABLE status, find the matching APPROVED request, mark request and copy ISSUED, insert the loan and commit. Any failed validation rolls the operation back, so no partial issue remains.

3.10 Return Workflow

Return uses the same scanning pattern as issue so that staff learn one consistent interaction. The administrator scans the student ID and the physical copy barcode. The backend verifies that an active, unreturned loan exists for that exact student and copy; matching only by title is never sufficient.

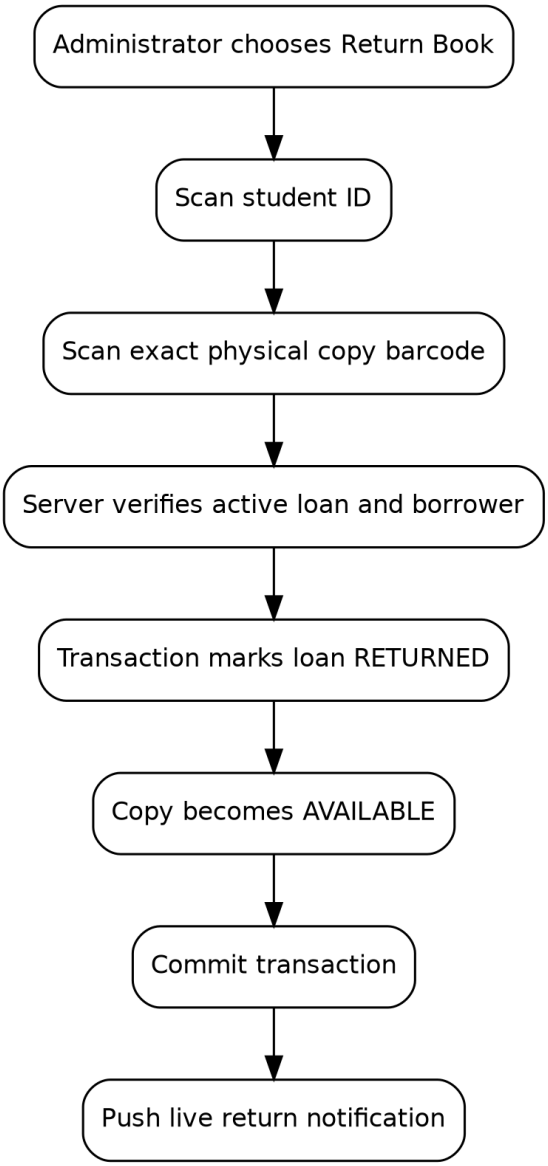


Fig. 3.7 Return book workflow

Inside one transaction the matching loan becomes RETURNED, return metadata is stored and the copy becomes AVAILABLE. The historical loan is retained, a return notification is created and availability is refreshed. A wrong borrower or unrecognized barcode is rejected without changing inventory.

3.11 Search, Recommendation and Deployment Design

Search is an in-memory service rebuilt from the real database. Active books are loaded together with total and available copy counts, then specialized structures are built for exact identifiers, tokens, prefixes, alphabetical ordering and relationships. A read/write lock allows queries to run safely while rebuild operations remain isolated. Availability can be refreshed for only the affected book after issue or return.

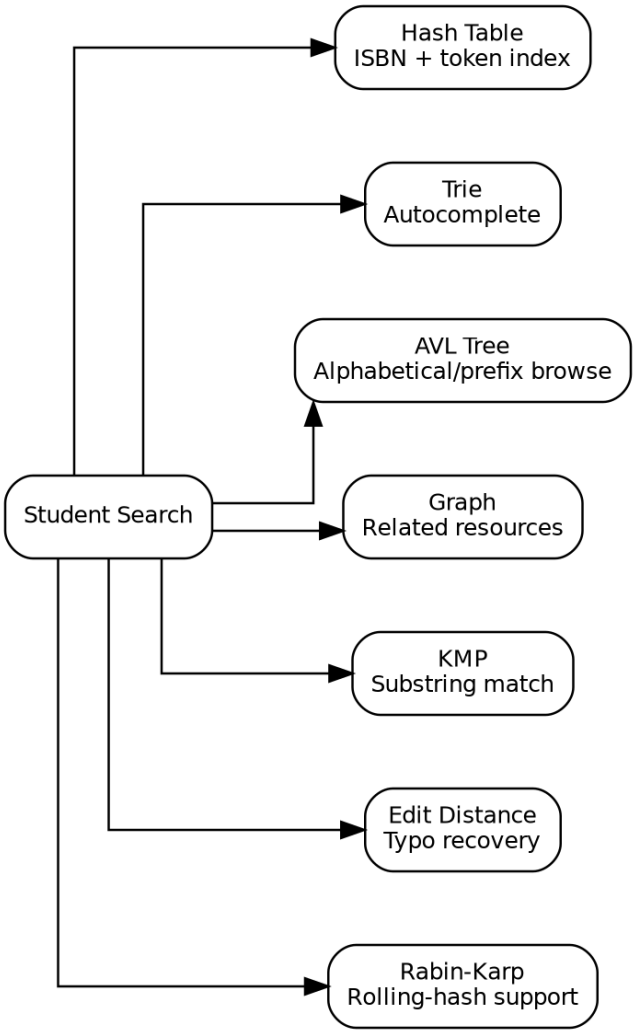


Fig. 3.8 DSA components used by the search subsystem

The system is deployed centrally: GitHub stores source, Render runs the Spring Boot service, Neon stores PostgreSQL data, and Windows clients communicate through HTTPS and WebSockets. Credentials and application secrets remain in environment variables, while all clients share the same server-side rules.

4. IMPLEMENTATION

4.1 Technology Stack and Project Structure

The implementation uses Java 21 across the desktop and backend to keep one primary programming language while still separating responsibilities. JavaFX 21 provides the desktop user interface. Spring Boot 3.5.x provides HTTP routing, dependency injection, validation, JDBC integration and WebSocket support. PostgreSQL is the durable store, Flyway applies versioned schema migrations, and Apache Commons CSV handles controlled import. The camera layer uses webcam-capture for Windows webcam access and ZXing for multi-format decoding.

Table 4.1 Technology stack

Component	Technology	Role
Desktop UI	JavaFX 21	Student/Admin screens, custom overlays, tables and responsive desktop interaction.
Camera access	webcam-capture 0.3.12	Enumerate cameras, configure resolution and capture frames.
Code decoding	ZXing 3.5.3	Decode QR, Code 128/39/93, EAN, UPC, ITF, Codabar, Data Matrix and PDF417.
Backend	Spring Boot 3.5.x / Java 21	REST, WebSocket, services, validation and deployment runtime.
Persistence	Spring JDBC + PostgreSQL	Queries, transactions, row locks and relational constraints.
Migrations	Flyway	Schema creation and controlled evolution.
Cloud database	Neon PostgreSQL	Central durable database.
Cloud hosting	Render Web Service + Docker	Public backend independent of client PCs.
Build/versioning	Maven + GitHub	Dependency management, packaging, source control and deploy source.

The project is separated into client, server, documentation and scripts. This makes it possible to build the Windows app independently from the cloud service. Deployment configuration is kept in the server module, while local development scripts and Windows packaging scripts remain outside the user-visible application.

4.2 Camera Scanner Implementation

The camera scanner is implemented as an IntelliLib overlay rather than as a separate camera program. When opened, it asynchronously enumerates webcams, selects the first available device and displays a camera picker only if more than one device exists. The preferred view size is HD when supported, with VGA fallback. Frames are captured on a daemon scheduled executor so camera work does not block the JavaFX application thread.

Each captured BufferedImage is shown in the JavaFX ImageView and is also passed to ZXing. The decoder uses HybridBinarizer, TRY_HARDER and an explicit set of supported formats: QR Code, Code 128, Code 39, Code 93, EAN-13, EAN-8, UPC-A, UPC-E, ITF, Codabar, Data Matrix and PDF417. Capture/decode attempts occur at roughly 110 ms intervals. When a code is recognized, an atomic flag prevents duplicate callbacks, the UI displays “Code recognized,” the overlay closes and the decoded text is passed to the calling workflow.

Pseudocode 4.1 In-app code scanner

```
OPEN_SCANNER(title, subtitle):
  cameras <- discoverWebcams()
  if cameras empty: show camera error; return
  open selected camera at HD if supported else VGA
  repeat every short interval until closed:
      frame <- camera.getImage()
      display frame inside IntelliLib
      value <- ZXing.decode(frame, supportedFormats, TRY_HARDER)
      if value found and not already completed:
          close camera
          callback(trim(value))
```

Camera permission and camera contention are handled with clear retry messages. Because the scanner is reusable, the same component is used for student login, student enrollment, student verification during issue/return and book-copy barcode scanning.

4.3 Student Authentication and Session Handling

Student authentication begins with the decoded card value. The backend first normalizes the scanned text to remove surrounding whitespace and invisible Unicode characters that can appear in scanner payloads. It then calculates HMAC-SHA256 using APP_CARD_PEPPER and queries for an active STUDENT user with the resulting card_hash. If no row matches, the server returns an actionable message explaining that the college ID has not been enrolled or the account is disabled. This behaviour was observed during live testing and is intentionally different from silently creating a user.

On successful match, AuthService creates a session. A 32-byte random token is generated, but only its SHA-256 hash is inserted into the sessions table together with the user ID, workstation identifier and expiry time. Student sessions expire after two hours. The raw token exists only on the authenticated client and is supplied as X-Session-Token on later requests. The server hashes the received token again and joins sessions to users to verify that the session is unexpired and the account remains active.

Pseudocode 4.2 Student card login

```
LOGIN_BY_CARD(rawCode,                                    workstation):
  code                      <-                      normalize(rawCode)
  if                code                empty:                reject
  cardHash        <-        HMAC_SHA256(APP_CARD_PEPPER,        code)
  student  <-  find  active  STUDENT  where  card_hash  =  cardHash
  if    not    found:    reject    as    not    enrolled/disabled
  sessionToken          <-          secureRandom(32          bytes)
  store    SHA256(sessionToken),    userId,    workstation,    expiry
  return sessionToken + student profile
```

4.4 Administrator Authentication and Management

Administrator login uses email and password because administrative privileges require a stronger credential than possession of a student card. The backend searches for an active ADMIN user by normalized email and verifies the supplied password against a BCrypt hash. The BCrypt encoder is configured with a work factor of 12. Administrator sessions are created through the same token mechanism as student sessions but use a longer expiry window of eight hours.

Student enrollment is performed from the Administrator → Students screen. The administrator can choose “Register one student,” scan the real college ID and complete the student details. For card systems where the QR payload is the student ID itself, the implementation can use that value as both student_code and card code, while still storing only the derived card hash. Bulk student import uses CSV with required student_code and name and optional card_code; if card_code is omitted for this card model, student_code is used. Duplicate student codes and duplicate card ownership are rejected.

The administrator can also activate or deactivate a student without deleting historical data. Deactivation immediately prevents card login because authentication requires active=TRUE. This is preferable to deleting the user, because borrow requests, loans, notifications and audit history remain linked to the correct identity.

4.5 Hash-Based Search Indexes

The SearchEngine loads active books and constructs an ISBN hash index and a token hash index. The ISBN index maps a normalized ISBN directly to the internal array position of the book. The token index maps normalized tokens to a compact list of books containing that token in title, author, category or keywords. Exact ISBN lookup therefore avoids scanning the catalogue, while token lookup quickly produces a candidate set for ranking.

When a query arrives, the normalized full query is first checked against the ISBN index. Matching tokens then contribute scores to candidate books. The implementation currently gives a high score to exact ISBN matches, accumulates token-match scores, and later adds bonuses for exact/prefix title matches, exact author matches, KMP substring matches and a bounded rating contribution. The numeric weights are implementation choices, while the DSA principle is the separation of candidate retrieval from final ranking.

Pseudocode 4.3 Hash-indexed candidate retrieval

```
REBUILD_INDEXES(books):
  for each book at index i:
      if ISBN exists: isbnIndex[normalize(ISBN)] <- i
      for token in tokenize(title, author, category, keywords):
          tokenIndex[token].addUnique(i)

SEARCH(query):
  q <- normalize(query)
  if isbnIndex contains q: score(book) += exactISBNWeight
  for token in q:
      for book in tokenIndex[token]: score(book) += tokenWeight
  rank candidate books with additional title/author/text signals
```

The custom hash structures are rebuilt from the database whenever a full rebuild is required. PostgreSQL remains the durable source of truth; the hash indexes are performance-oriented in-memory structures and can be reconstructed after process restart.

4.6 Trie and AVL Indexes

Autocomplete is implemented using a Trie containing book titles and author names. A Trie stores strings by shared prefixes, so the characters already typed by the user identify a subtree containing possible completions. This directly matches the interaction expected from a search field: after two or more characters, the client requests a bounded list of suggestions and presents them beneath the search box. Selecting a suggestion immediately performs the search.

Alphabetical and prefix-style title browsing is supported by an AVL book index. An AVL tree is a self-balancing binary search tree, so insertion maintains a bounded height through rotations. Ordered traversal therefore remains efficient even when books are not inserted in sorted order. In IntelliLib the AVL structure stores title ordering independently from the hash/token structures, demonstrating that different access patterns benefit from different data structures.

Pseudocode 4.4 Trie suggestion and AVL insertion

```
TRIE_INSERT(text):
  node                              <-                              root
  for         character         c         in         normalize(text):
      node                  <-                  node.childOrCreate(c)
  node.terminal                         <-                          true

AVL_ADD(title,                                                 bookIndex):
  root    <-    insertByNormalizedTitle(root,    title,    bookIndex)
  update                                                          height
  if        balance        factor        outside        [-1,        1]:
      apply required single/double rotation
```

These indexes are deliberately hidden from the ordinary interface. A student experiences only “suggestions” and ordered search behaviour, while the report and viva can explain why a Trie and a balanced tree are appropriate underneath those features.

4.7 Graph-Based Related Resources and Recommendations

Related Books is modelled using an undirected graph whose vertices correspond to books. During search-engine rebuild, books are placed into temporary buckets for normalized category, author and individual keywords. Books sharing a bucket are connected, with a cap on bucket processing to prevent extremely common values from creating an unnecessarily dense graph. A related-book request therefore becomes a bounded neighbour lookup rather than a new full-catalogue comparison.

Recommendations are intentionally different from related-book edges. “Popular in the library” is derived from successful loan history, while “Recommended for you” uses only the authenticated student’s own activity such as viewed books, borrowing history and content relationships. The system does not claim to use an external AI model. When history is insufficient, the client displays an honest empty state. This behaviour is more defensible than presenting a random set of resources under an artificial personalization label.

Pseudocode 4.5 Related-resource graph construction

```
BUILD_GRAPH(books):
  buckets <- group book indexes by category, author and each
keyword
  for each bucket:
      take bounded number of indexes
      connect each pair inside the bounded set

RELATED(bookId, limit):
  i <- idToIndex[bookId]
  neighbours <- graph.neighbors(i, limit)
  return book summaries for neighbours
```

The graph provides an intuitive academic explanation: resources are not only isolated records; they form relationships through shared metadata. This design can later be extended with weighted edges derived from co-borrowing or explicit subject ontologies without changing the public interface.

4.8 String Matching and Typo Recovery

After candidates are obtained from exact and token indexes, IntelliLib applies additional text-processing techniques. KMP is used to test whether the full user query occurs inside a book's searchable metadata without repeatedly restarting comparisons after each mismatch. Edit distance is used as a bounded recovery mechanism when normal token lookup yields no candidates. The implementation compares a short query against words in titles and authors and permits only a small number of edits, avoiding an expensive unrestricted fuzzy search across long text.

Rabin-Karp is also included in the DSA module as rolling-hash string-matching support. It is useful academically because it demonstrates a different strategy from KMP: instead of using a prefix/failure function, it compares hashes of windows and verifies potential matches when hashes agree. The current search ranking relies primarily on the hash/token index, KMP and edit distance, while Rabin-Karp remains available for hash-based pattern matching extensions.

Table 4.2 DSA roles and complexity summary

Technique	Use in IntelliLib	Typical complexity idea
Hash Table	ISBN and token candidate lookup	Average O(1) lookup per key.
Trie	Title/author autocomplete	O(p) traversal for prefix length p plus output.
AVL Tree	Sorted title access	O(log n) search/insert due to balancing.
Graph	Related resources	Neighbour lookup proportional to connected edges.
KMP	Substring occurrence check	O(n + m) text and pattern processing.
Edit Distance	Small typo recovery	Dynamic programming O(a × b) for compared strings.
Rabin-Karp	Rolling-hash pattern support	Average linear scanning with rolling hash and verification.

The combined design avoids treating one algorithm as universally best. Exact identifiers, prefixes, ordered browsing, relationships and typo handling are different access problems, so IntelliLib applies the structure that fits each problem.

4.9 Requests, Notifications and Real-Time Updates

A borrow request begins only if at least one physical copy of the selected book is currently AVAILABLE. The service also checks whether the same student already has an active REQUESTED or APPROVED request for the same book. A successful request inserts a REQUESTED row and publishes a WebSocket message for the administrator request topic. This lets an administrator screen refresh without requiring repeated manual polling.

Approval is also guarded by availability. Before approving, the service counts available copies and already approved reservations for the same book so that approvals do not exceed the unreserved inventory. Approved requests receive an expiry approximately two days in the future and the student receives a notification indicating that the book is ready for collection. Rejection stores an optional note and produces a corresponding notification.

Pseudocode 4.6 Borrow request approval

```
APPROVE(requestId,                                                admin):
  request          <-          lock          request           row
  require           request.status           =           REQUESTED
  available   <-   count   AVAILABLE   copies   for   request.book
  reserved <- count unexpired APPROVED requests for request.book
  require       available       -       reserved       >=       1
  request.status                      <-                    APPROVED
  request.expires_at       <-       now       +       approvalWindow
  save                       decision                       metadata
  create student notification
```

Notifications are stored in PostgreSQL so they remain visible after reconnecting. WebSocket delivery is therefore an immediacy mechanism, not the sole copy of the message. This distinction is important for reliability: if a student app is offline at the moment of approval, the notification still exists in the database when the student reconnects.

4.10 Atomic Circulation, Import, Migrations and Packaging

Issue and return operations use Spring transactions and SELECT ... FOR UPDATE locking on the relevant records. The issue flow re-verifies the student, locks the copy, verifies AVAILABLE status and an unexpired APPROVED request, then updates copy and request state and inserts the loan. The return flow verifies an active loan matching both student and copy, marks the loan RETURNED and restores the copy to AVAILABLE. Database constraints further prevent multiple active loans for the same copy.

Real library data enters through CSV import. Student import validates student_code and name, derives or accepts a card code, rejects card collisions and updates existing students safely. Book import creates or updates bibliographic metadata, while copy import assigns exact barcodes to book records. Flyway migration scripts create the initial schema and later extensions such as book views and unique active-loan constraints, making new cloud databases reproducible.

Deployment and packaging complete the production path. The backend Dockerfile builds the Spring Boot JAR with Maven and runs it on a Java 21 JRE. To avoid shared-host Maven Central throttling encountered during deployment, the build was changed to use Google's hosted Maven Central mirror. Render provides the public web service and forwards traffic to the PORT environment value. On Windows, a packaging script uses jpackage to create IntelliLib.exe together with app and runtime directories. End-user PCs therefore launch the application directly without Maven, Docker or a separately installed JDK.

5. RESULTS

5.1 Windows Build and Packaging

The JavaFX desktop client was compiled successfully after correcting source-level issues found during the first packaging attempt. The final portable build process completed and produced client\dist\IntelliLib\IntelliLib.exe together with the app and runtime directories. This verifies that the application can be distributed as a self-contained Windows app-image rather than requiring a command-line Maven launch on every library PC.

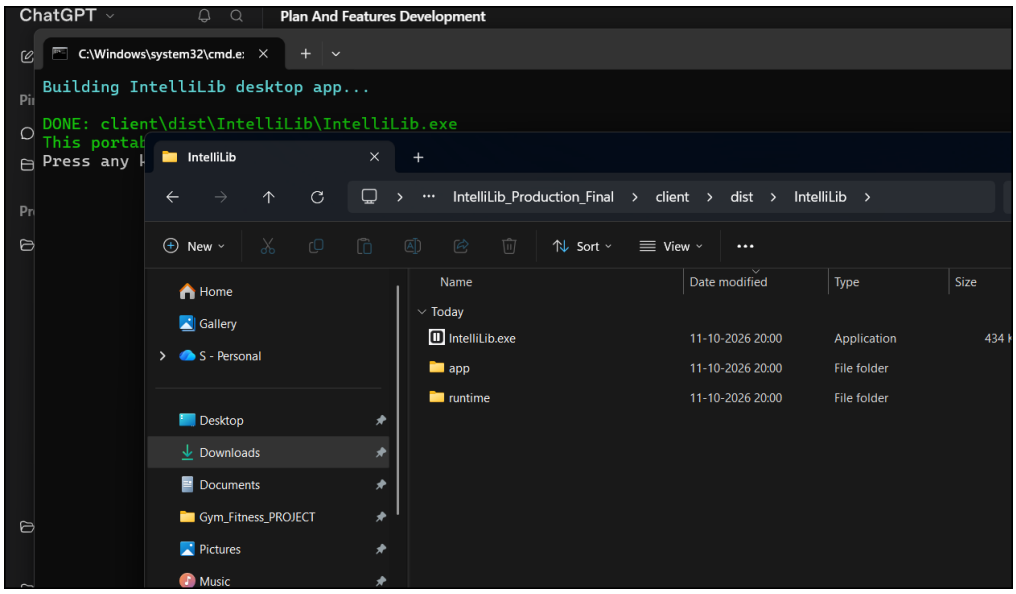


Fig. 5.1 Portable Windows build output containing IntelliLib.exe, app and runtime

The packaged application was launched directly from the generated directory and opened the IntelliLib landing interface. Because the runtime is bundled, copying the whole IntelliLib directory is sufficient for another compatible Windows PC. The package still remains unsigned, so Windows SmartScreen can warn on a newly built executable; code signing is identified as a future institutional deployment improvement rather than bypassed in the design.

5.2 Camera Scanning and Validation

The in-app camera overlay successfully opened a connected USB webcam and displayed a live preview inside IntelliLib. The college ID QR code was recognized by ZXing, confirming that the scanner pipeline from webcam frame to decoded payload is functional. The identifying card region in Fig. 5.2 is intentionally pixelated in this report to avoid exposing personal details, while the surrounding interface demonstrates the integrated camera workflow.

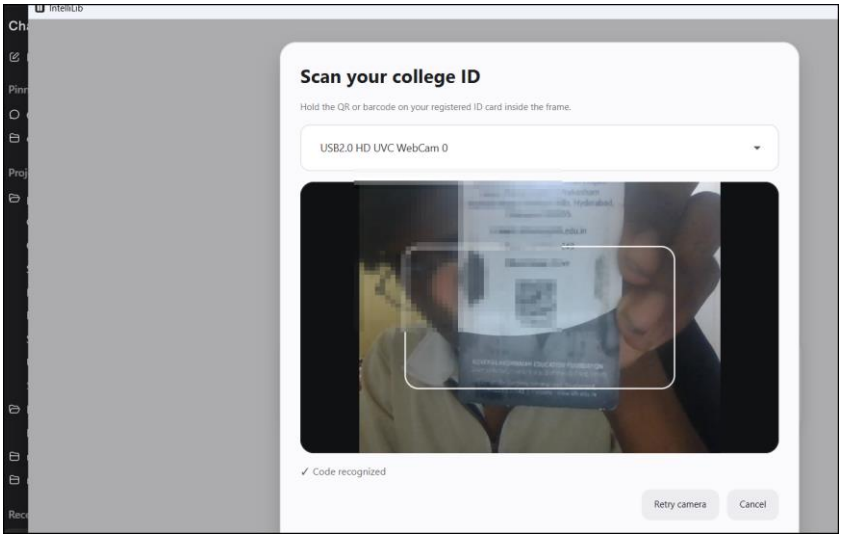


Fig. 5.2 In-app camera scanner with privacy-redacted college ID

When the decoded card had not yet been enrolled in the live database, the backend correctly rejected authentication with an explicit "not enrolled or disabled" message. This is a positive validation result: a syntactically valid QR code cannot create a session by itself. Enrollment must occur through the administrator workflow, after which the stored card hash can match the scanned value. The same scanner component is reused for issue/return identity checks and physical-copy barcodes.

5.3 Cloud Deployment

The Spring Boot backend was deployed to Render from the project GitHub repository and connected to the Neon PostgreSQL database. The successful deployment logs show the application starting on the Render-assigned port, Hikari connection-pool initialization, Flyway validation of the schema, WebSocket broker startup and the Render "service is live" message. The public backend address can therefore be used by the Windows desktop client instead of localhost.

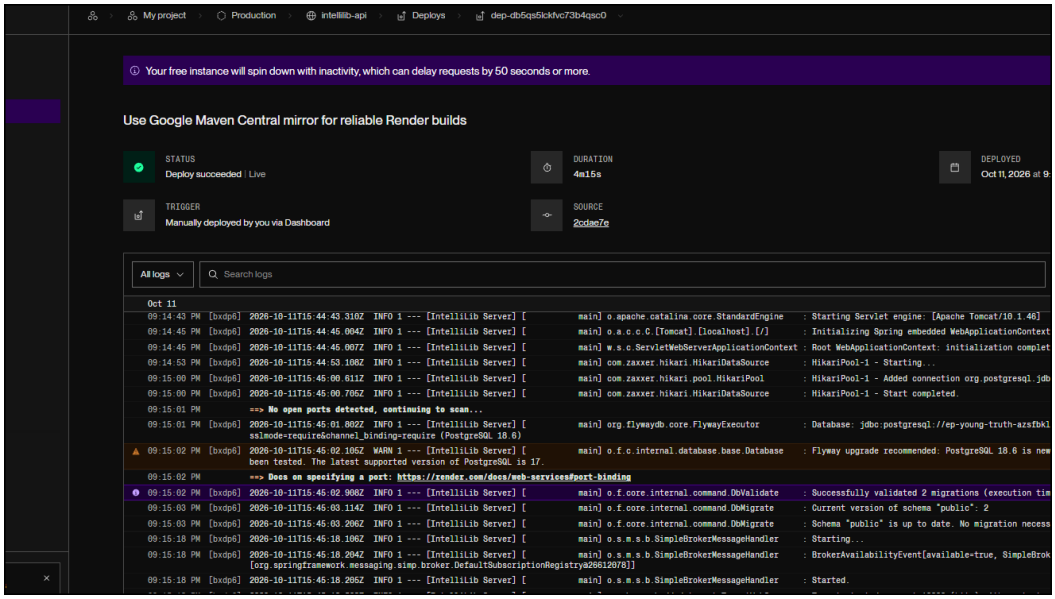


Fig. 5.3 Successful Render deployment connected to Neon PostgreSQL

Deployment also exposed a real infrastructure issue: Maven Central returned HTTP 429 responses to Render's shared build environment. Retrying did not reliably solve the problem, so the Docker build was updated to use Google Cloud's Maven Central mirror. The subsequent deployment succeeded. This result is important because it demonstrates not only that the application runs locally, but that the project has a repeatable path to a cloud-hosted backend independent of the development PC.

5.4 Live Database and Verification Matrix

After the cloud service became live, the administrator could sign in and open the Library Overview dashboard. The initial production database showed zero books, zero physical copies, zero students, zero issued loans, zero pending requests and zero overdue loans. This is the intended result because no generated production records are inserted. The screen therefore proves that the UI is reading live database aggregates rather than hard-coded demonstration values.

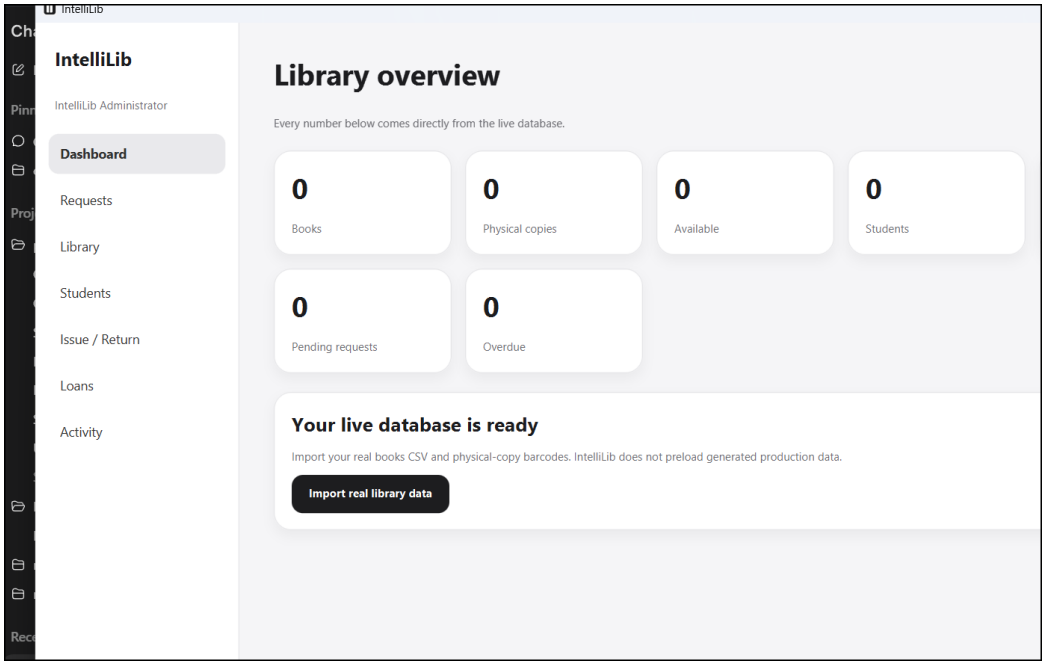


Fig. 5.4 Administrator dashboard using live database counts

Table 5.1 Verified implementation results

Test / observation	Result	Status
Windows portable build	IntelliLib.exe with bundled app/runtime produced successfully.	Verified
Application launch	Packaged desktop application opened successfully.	Verified
Webcam access	USB webcam discovered and preview displayed inside IntelliLib.	Verified
QR decode	College ID QR decoded and scanner reported code recognized.	Verified
Unenrolled-card protection	Server rejected a real decoded card that was not registered.	Verified
Cloud API deployment	Render reported the service live.	Verified
Neon connectivity	Spring Boot connection pool and Flyway schema validation succeeded.	Verified
Live dashboard	Administrator saw zero real records from the new production database.	Verified
Full request → issue → return with real library data	Requires enrollment/import of real students/books/copies.	Pending pilot data

6. CHALLENGES AND LIMITATIONS

6.1 Development Challenges and Solutions

The development process revealed several issues that would not appear in a purely theoretical DSA demonstration. The first Windows packaging attempt exposed Java compilation errors in the desktop client, including an uninitialized callback reference and a webcam-resolution constant not provided by the selected library version. These were corrected and the client subsequently produced a working portable app-image. A backend compilation error around CSV export also appeared during cloud deployment and was fixed by handling IOException correctly.

Table 6.1 Challenges and applied solutions

Challenge	Cause	Applied solution
Student QR recognized but login rejected	The card decoded correctly, but no matching enrolled student existed in the live database.	Keep secure server rejection; require administrator enrollment of the real card before login.
JavaFX build failed	Callback capture and unsupported HD720 constant.	Use initialized callback holder and supported WebcamResolution.HD.
Backend CSV compile failure	IOException was not handled inside JDBC callback.	Catch/wrap the checked exception and stop build scripts on failure.
Render Maven HTTP 429	Shared cloud build IP was rate-limited by Maven Central.	Use Google-hosted Maven Central mirror in the Docker build.
Localhost dependency	Backend initially ran on developer machine.	Deploy Spring Boot to Render and PostgreSQL to Neon; configure client with public URL.
Fake demo data risk	Generated catalogues can make a project look complete while hiding real-data issues.	Production database starts empty; import/enroll only real records.

These issues improved the final architecture because each fix was incorporated into the build or runtime design instead of treated as a one-time manual workaround. The project therefore records both successful features and the engineering decisions required to make them repeatable.

6.2 Current Limitations and Risk Management

The zero-cost deployment has limitations. Render Free can suspend an inactive web service, so the first request after a long idle period may experience a cold-start delay. This is acceptable for an academic demonstration and pilot but is not equivalent to a paid institutional service-level agreement. A production college deployment should use an always-on plan or an institution-managed server with monitoring, backups and defined recovery procedures.

Card-based authentication also has a security boundary. A QR code that contains only a student number proves possession of the card, not the identity of the person presenting it. For a library desk this can be mitigated by displaying the student name and institutional photo during high-value operations, integrating with the official identity provider, or requiring an additional factor. IntelliLib already avoids storing raw card codes, but the physical-card trust model must be understood rather than overstated.

The current live database deliberately contains no real library catalogue until authorized data is imported. Consequently, full end-to-end request, approval, issue and return testing with actual book copies remains a pilot activity after student/book/copy enrollment. The search structures are rebuilt in memory from the database; extremely large institutional catalogues may eventually benefit from incremental index maintenance or a dedicated search service. Finally, Windows app signing, automated backups, formal accessibility testing, institutional privacy review and load testing are future production-hardening tasks.

7. CONCLUSION AND FUTURE WORK

7.1 Conclusion

IntelliLib demonstrates that Data Structures and Algorithms can be applied inside a real software workflow rather than displayed only as isolated examples. The project began with the need to search a large collection efficiently and evolved into a complete two-role physical-library system. Students authenticate through an in-app scan, search real catalogue data, request resources and receive status updates. Administrators manage real users and inventory, make request decisions and perform issue/return operations using the exact student identity and physical copy barcode.

The implementation combines multiple DSA techniques because the library domain contains multiple access patterns. Hash tables support exact and token lookups, a Trie supports prefix suggestions, an AVL tree supports ordered access, a graph represents related books, KMP supports substring matching and edit distance supports small spelling corrections. These structures are rebuilt from PostgreSQL, which remains the durable source of truth. Database transactions and constraints protect circulation integrity, while Spring Boot and WebSockets provide centralized business logic and live communication.

The completed work also demonstrates production-oriented engineering beyond the algorithms themselves. The desktop client can be packaged as IntelliLib.exe with a bundled runtime; the backend has been deployed on Render; the database runs on Neon; Flyway reproduces the schema; the webcam scanner successfully reads the college card; and the server correctly rejects an unenrolled scanned identity. The live administrator dashboard starts with real zero values rather than fabricated data. These results show that the project has progressed from a classroom concept to a working cloud-connected system that is ready for real catalogue and student data during the pilot phase.

7.2 Future Work

The immediate next step is operational pilot testing with authorized college data. Real student records should be imported or enrolled, real bibliographic records should be loaded, and every physical copy should receive or map to a unique barcode. The team can then execute a controlled request → approval → issue → return cycle and record timing, error cases and staff feedback. This would convert the current implementation verification into a complete institutional acceptance test.

- Integrate with the institution's official identity provider or student-photo service so card presentation can be checked against a trusted identity source.
- Add code signing and a managed installer/update channel for Windows distribution across library PCs.
- Move from the free sleeping backend to an always-on institutional or paid cloud service with monitoring, health alerts, backups and restore drills.
- Add automated integration tests that run against a temporary PostgreSQL instance and exercise invalid state transitions, duplicate issue protection and return validation.
- Improve accessibility through keyboard navigation review, screen-reader labels, contrast testing and focus-state validation.
- Introduce richer recommendation weights using co-borrowing and subject metadata while preserving explainability and avoiding invented personalization when data is sparse.
- Add optional reservation queues, configurable loan policies and librarian-defined due-date rules once the core pilot workflow is stable.
- Implement incremental updates to the in-memory search indexes after catalogue changes so very large catalogues do not require a full rebuild for every update.
- Provide administrative backup/export tools and a documented disaster-recovery process for institutional use.

Future work should preserve the project's core principle: new features are justified only when they improve a real library workflow or strengthen correctness, security and maintainability. The goal is not to maximize the number of screens, but to make the system dependable enough to support actual students, books and library staff.

7.3 Skills Learned During Development

The project required the team to combine algorithmic reasoning with software-engineering discipline. On the DSA side, we learned to choose a structure by access pattern rather than by familiarity: exact identifiers favour hashing, prefixes favour tries, ordered keys favour balanced trees, relationships favour graphs, and approximate text requires string-processing techniques. We also learned that asymptotic efficiency alone is not sufficient; an index must be rebuilt safely, synchronized during concurrent access and integrated with persistent data.

On the application side, the team gained experience with JavaFX event handling, asynchronous work, custom overlays, webcam integration, QR/barcode decoding, REST clients, JSON serialization and WebSocket messaging. Backend work included Spring Boot controllers and services, authentication, password hashing, HMAC-based card mapping, random sessions, SQL joins, aggregates, transactions, row locking and error handling. Database work included schema design, foreign keys, indexes, partial unique constraints and Flyway migrations.

Deployment provided another set of practical lessons. We learned why localhost is not a production architecture, how environment variables separate secrets from source code, how Docker images create a repeatable server runtime, how Render exposes an application through a public HTTPS URL, and how Neon provides a central PostgreSQL database. Real deployment failures such as Maven repository throttling highlighted the importance of reading logs and correcting the build process rather than repeatedly changing unrelated parts of the system. Finally, documenting the project reinforced the need to distinguish verified results from planned tests and to avoid presenting generated data as evidence of production readiness.

REFERENCES

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 4th ed., MIT Press, 2022.

[2] R. Sedgewick and K. Wayne, Algorithms, 4th ed., Addison-Wesley, 2011.

[3] OpenJFX, "JavaFX Documentation," https://openjfx.io/, accessed 11 October 2026.

[4] VMware / Spring, "Spring Boot Reference Documentation 3.5," https://docs.spring.io/spring-boot/3.5/reference/, accessed 11 October 2026.

[5] The PostgreSQL Global Development Group, "PostgreSQL 18 Documentation," https://www.postgresql.org/docs/current/, accessed 11 October 2026.

[6] Render, "Web Services – Render Docs," https://render.com/docs/web-services, accessed 11 October 2026.

[7] Neon, "Neon Documentation," https://neon.com/docs, accessed 11 October 2026.

[8] ZXing Project, "ZXing: Multi-format 1D/2D barcode image processing library," https://github.com/zxing/zxing, accessed 11 October 2026.

[9] Redgate, "Flyway Documentation," https://documentation.red-gate.com/flyway, accessed 11 October 2026.

[10] OWASP Foundation, "Password Storage Cheat Sheet," https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html, accessed 11 October 2026.

[11] Apache Software Foundation, "Apache Commons CSV," https://commons.apache.org/proper/commons-csv/, accessed 11 October 2026.

[12] Koneru Lakshmaiah Education Foundation, "Project Documentation Format 2026-27" and "Guidelines for Project Document," Department of Computer Science and Engineering, 2026.

[13] IntelliLib project source repository, "KLH_CSE_2026-27_DSA-3_S8_TEAM2_LIBRARY_SYSTEM," team project repository, accessed during implementation and documentation.